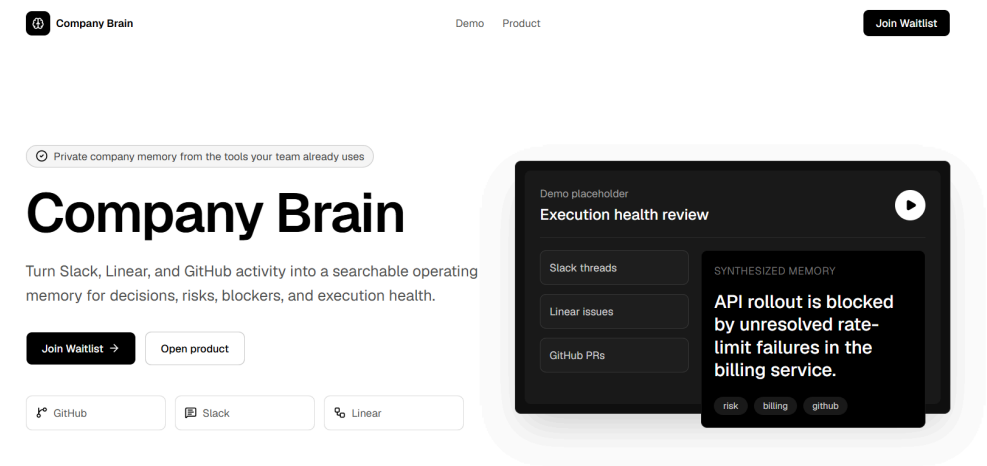


Company Brain

The organizational intelligence layer that never sleeps.

Connect your tools. Extract signal. Surface what leadership needs to know.

license not specified



Live KPIs, mental model status, and quick-action controls

[Quick Start](#) · [Why Company Brain?](#) · [Docs](#) · [PDF version](#) · [Contributing](#) · [Report Bug](#)

Table of Contents

- [Why Company Brain?](#)
- [Overview](#)
- [Features](#)
- [Screenshots](#)
- [Quick Start](#)
- [Installation](#)
- [Usage](#)
- [Architecture](#)
- [Configuration](#)
- [API Reference](#)
- [Contributing](#)
- [Roadmap](#)
- [Acknowledgments](#)

Why Company Brain?

The biggest blocker to AI automation of companies is no longer the models — they just got so good so quickly. Now the blocker is **domain knowledge**.

Every company has critical know-how scattered everywhere. Some of it lives in people's heads. Some of it is buried in old email accounts, Slack threads, support tickets, and databases. The company works because humans vaguely remember where that knowledge is and how to apply it.

But AI agents can't operate like that. If we want every company to run on AI automation, we need a new primitive: **a company brain**.

We need Garry's G-Brain, but for every business in the world. A system that pulls knowledge out of all these fragmented sources, structures it, keeps it current, and turns it into an executable skills file for AI.

This isn't a company-wide search or a chatbot over documents. It's a living map of how a company works: how refunds get handled, how pricing exceptions are decided, or how engineers respond to incidents.

Then AI systems can use that skills file to actually do the work safely and consistently.

The company brain becomes the missing layer between raw company data and reliable AI automation.

Every company in the world is going to need one.

Overview

Company Brain is that layer — a semantic memory system for organizations. It continuously ingests activity from connected tools (Slack, GitHub, Linear), uses Gemini to extract operational signals (decisions, risks, blockers, milestones), stores them in a persistent vector memory bank via Hindsight, and runs periodic reflection passes to synthesize leadership-grade observations.

Built for engineering and product leaders who need a real-time pulse on what's happening across their teams — without reading every message.

- Multi-tenant, per-company memory isolation

Gemini-powered AI extraction (OpenAI-compatible)

MCP proxy endpoint for Cursor / Claude

Features

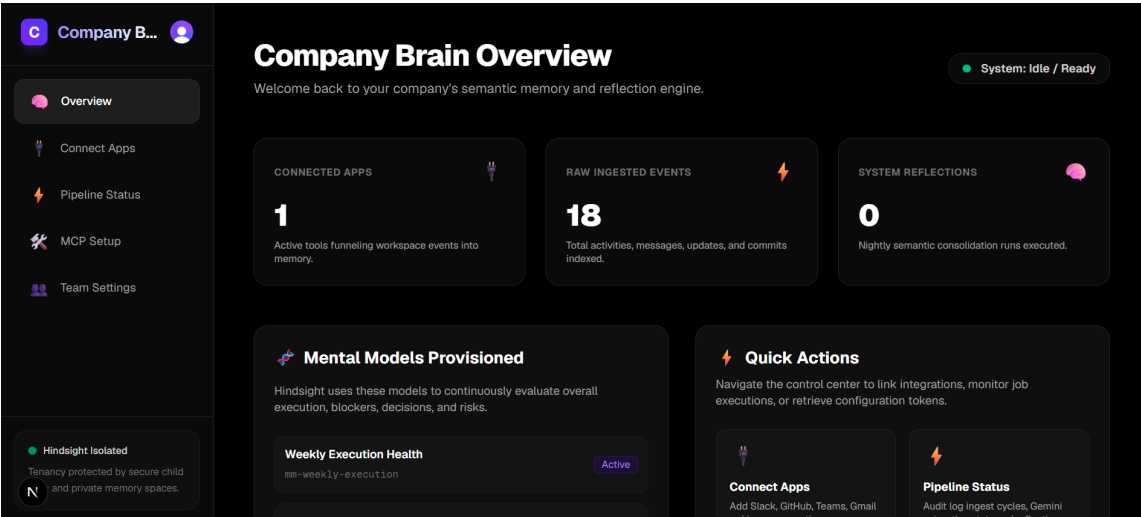
	Feature	Description
	Multi-App Ingestion	Connects Slack, GitHub, and Linear via Composio OAuth
	AI Signal Extraction	Gemini 2.0 Flash classifies every event into facts, decisions, risks, action items, and milestones
	Persistent Vector Memory	All extracted knowledge is retained in a Hindsight Memory Bank with tenant isolation
	Nightly Reflection	Scheduled synthesis pass queries the bank with leadership questions and re-retain observations
	MCP Integration	Token-authenticated proxy exposes Hindsight MCP to Cursor, Claude Desktop, and ChatGPT

	Mental Models	Four standing mental models: Weekly Execution Health, Active Blockers, Recent Decisions, Engineering Risks
	Scoped API Keys	Per-company read-only Hindsight keys with configurable TTL for direct MCP access
	Pipeline Dashboard	Full audit log of every ingestion run, extraction metric, and reflection cycle
	Multi-Tenant Auth	Clerk organization-based authentication with webhook-based member sync
	Deduplication	SHA-256-keyed upserts ensure no event is extracted twice, even across overlapping ingestion windows

Screenshots

Dashboard Overview

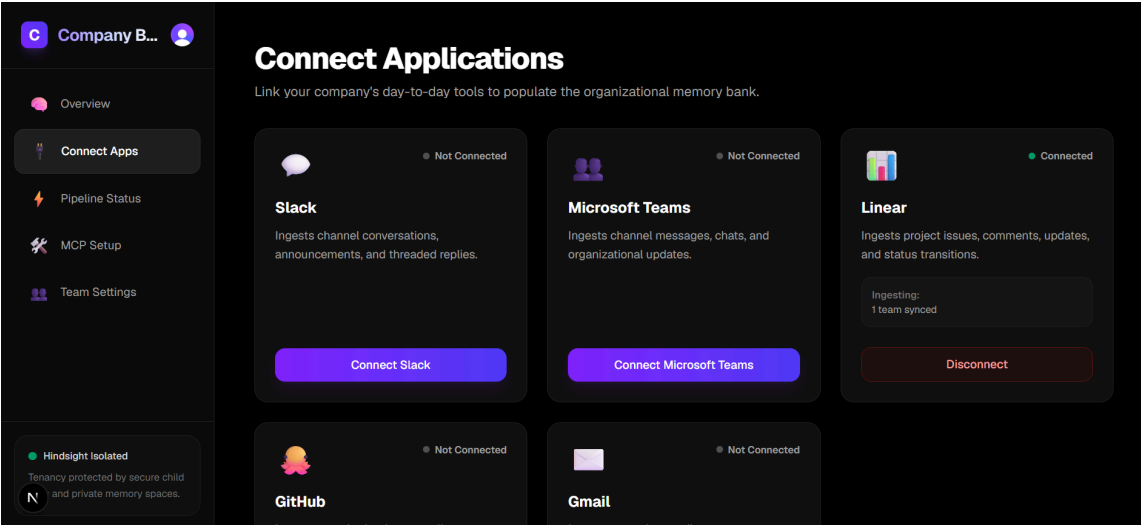
The main control center — live sync status, KPI counters, mental model roster, and quick-action navigation.



Connected Apps · Raw Events · System Reflections — all at a glance

Connect Apps

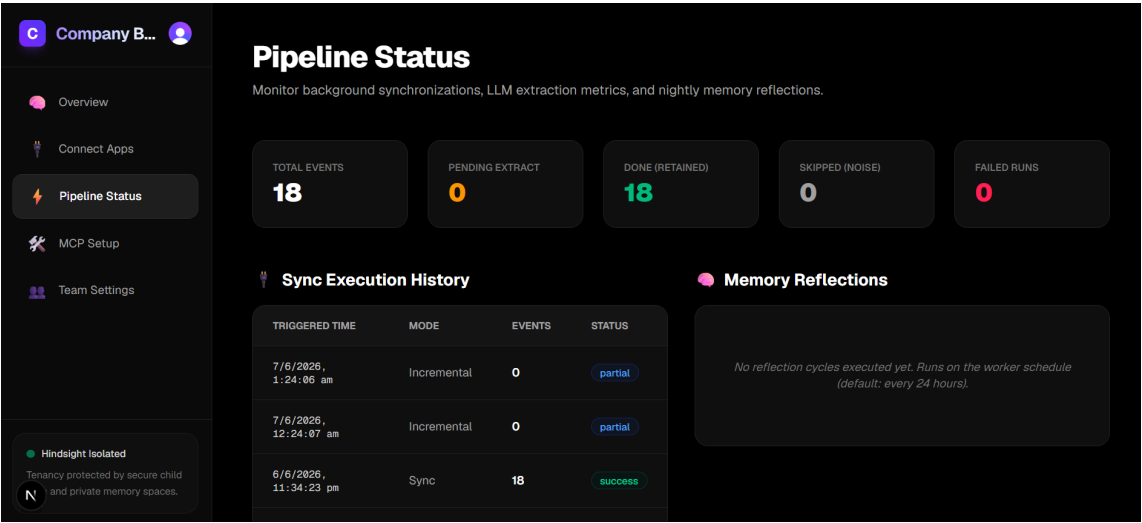
OAuth integration manager — connect or disconnect Slack, GitHub, and Linear. GitHub sync requires selecting repositories.



Connect your workspace tools to begin ingesting organizational activity

Pipeline Status

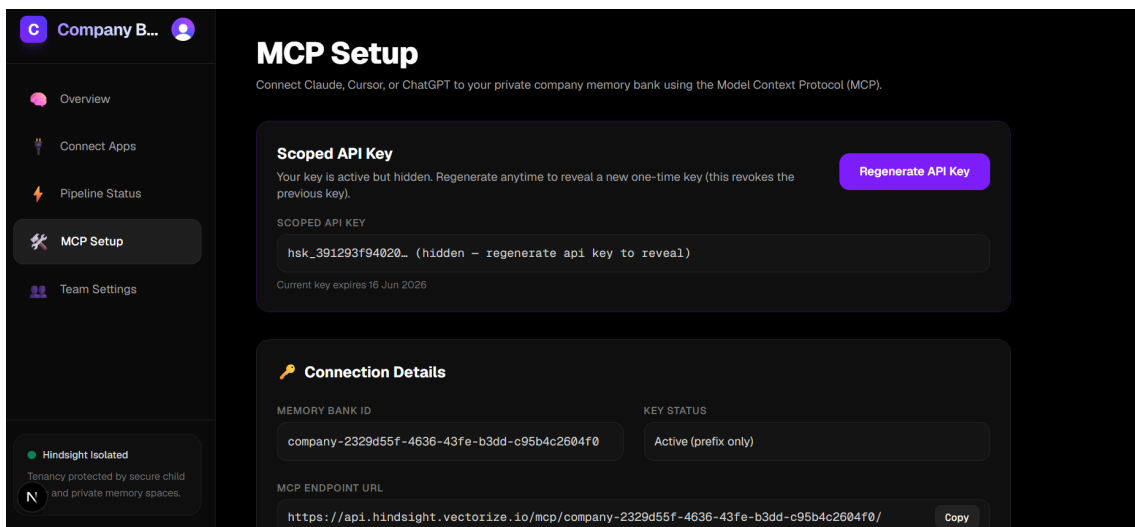
Full observability into every ingestion run, Gemini extraction metric, and nightly reflection cycle.



Extraction metrics (pending / done / skipped / failed) alongside sync and reflection history

MCP Configuration

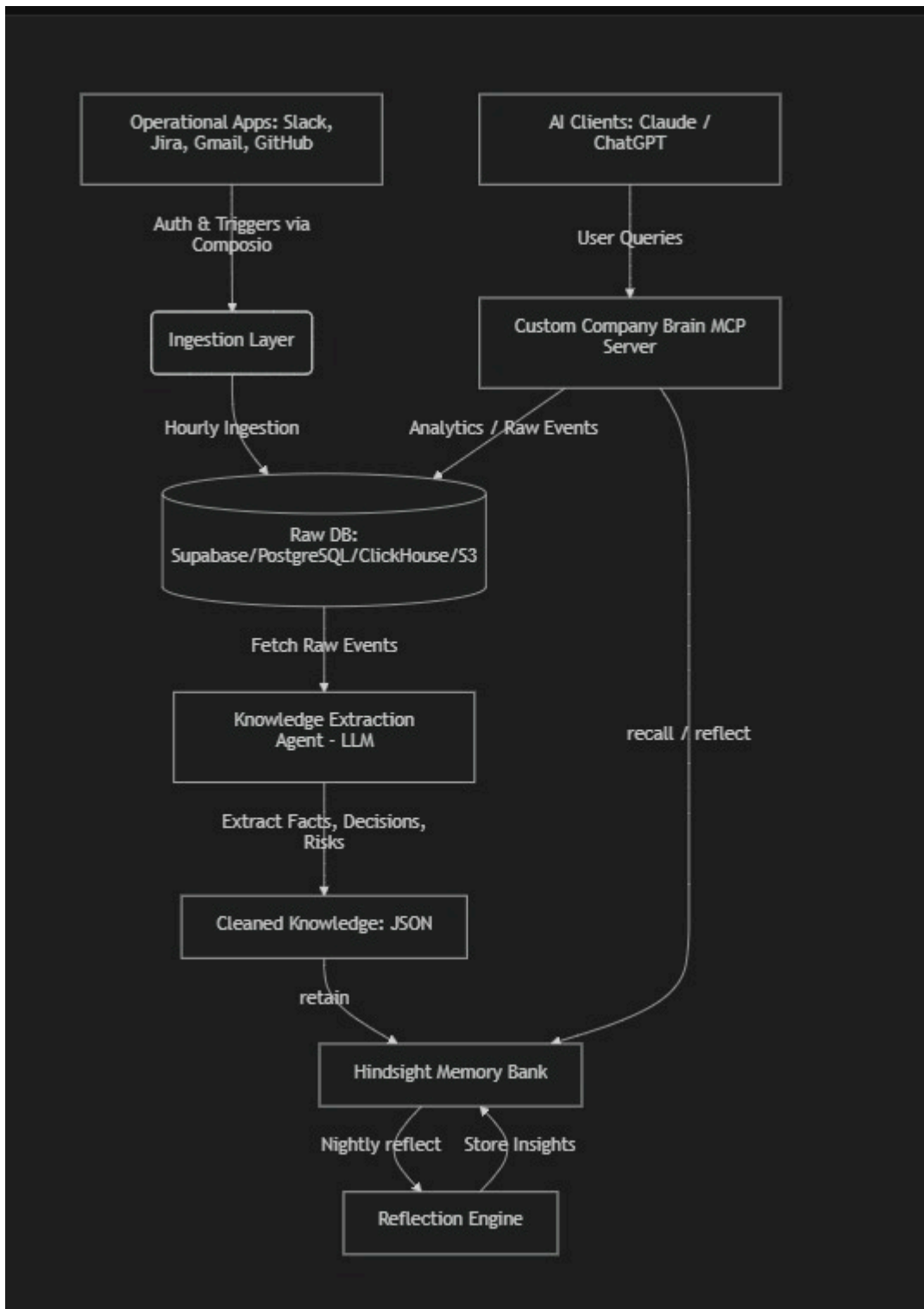
Copy the token-based MCP URL for Claude or ChatGPT, or generate a scoped Hindsight API key for direct access.



One-click URL copy and scoped key generation for AI IDE integration

Onboarding

Guided setup flow — create or join a Clerk organization, provision the Hindsight Memory Bank, and generate the initial scoped API key.



New organizations are provisioned with a Hindsight bank and four mental models in one step

Quick Start

```
# 1. Clone and install dependencies
git clone https://github.com/harshkansal031/company-brain.git
cd company-brain
pnpm install

# 2. Copy env template and fill in your keys (see Configuration section)
cp .env.example .env

# 3. Apply database migrations
pnpm db:migrate

# 4. Start the web app and worker in separate terminals
pnpm dev      # Terminal 1 – Next.js on http://localhost:3000
pnpm worker   # Terminal 2 – pipeline + reflection worker
```

Sign up, complete onboarding, connect at least one tool on `/connect` , and the worker will begin ingesting on its configured interval.

Installation

Prerequisites

Tool	Minimum Version	Notes
Node.js	20.x	LTS recommended
pnpm	9.x	<code>npm install -g pnpm</code>
Supabase project	—	Free tier works; needs Postgres connection strings
Clerk account	—	Organization feature must be enabled
Composio account	—	For OAuth tool connections
Hindsight Cloud account	—	ui.hindsight.vectorize.io — enable "Allow creating scoped child keys" on your parent API key
Gemini API key	—	Google AI Studio — <code>gemini-2.0-flash</code> is the default model

Install from source

```
git clone https://github.com/harshkansal031/company-brain.git
cd company-brain
pnpm install
cp .env.example .env
# Edit .env with your credentials (see Configuration section)
pnpm db:migrate
pnpm build
```

Production

Run the Next.js server and worker as separate processes (e.g. systemd, Docker, or a process manager):

```
pnpm build
pnpm start      # Web application
pnpm worker     # Pipeline worker (separate process)
```

Configure `CRON_SECRET` in production so `/api/cron/*` routes require a Bearer token.

Usage

Running a manual pipeline

The worker runs pipelines automatically on `PIPELINE_INTERVAL_MS` (default: 1 hour). Trigger one manually via HTTP:

```
curl -X POST http://localhost:3000/api/cron/pipeline \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $CRON_SECRET" \
-d '{ "companyId": "your-company-uuid" }'
```

Response:

```
{
  "success": true,
  "runId": "abc123...",
  "extraction": {
    "processed": 42,
    "skipped": 8,
    "failed": 0
  }
}
```

In development, the `Authorization` header is optional. In production, `CRON_SECRET` is required.

Worker schedule

The worker in `worker/index.ts` runs on startup and repeats on configured intervals:

```
// On startup: immediately run pipelines for all active companies
void runScheduledPipelines();

// Repeat every PIPELINE_INTERVAL_MS (default: 1 hour)
setInterval(() => void runScheduledPipelines(), PIPELINE_INTERVAL_MS);

// Repeat every REFLECTION_INTERVAL_MS (default: 24 hours)
setInterval(() => void runScheduledReflection(), REFLECTION_INTERVAL_MS);
```

For faster iteration during development, add `.env.local` :


```
PIPELINE_INTERVAL_MS=60000      # Run pipeline every 60 seconds
REFLECTION_INTERVAL_MS=120000   # Run reflection every 2 minutes
```

MCP setup in Cursor or Claude Desktop

- 1. Complete onboarding and open /mcp
- 2. Copy the **Connect to Claude / ChatGPT** URL (/api/mcp/{token}/sse)
- 3. Add it as an MCP server in your AI client — no separate API key required

Alternatively, generate a scoped Hindsight API key on the same page and use the direct Hindsight MCP URL with that key.

Extraction pipeline

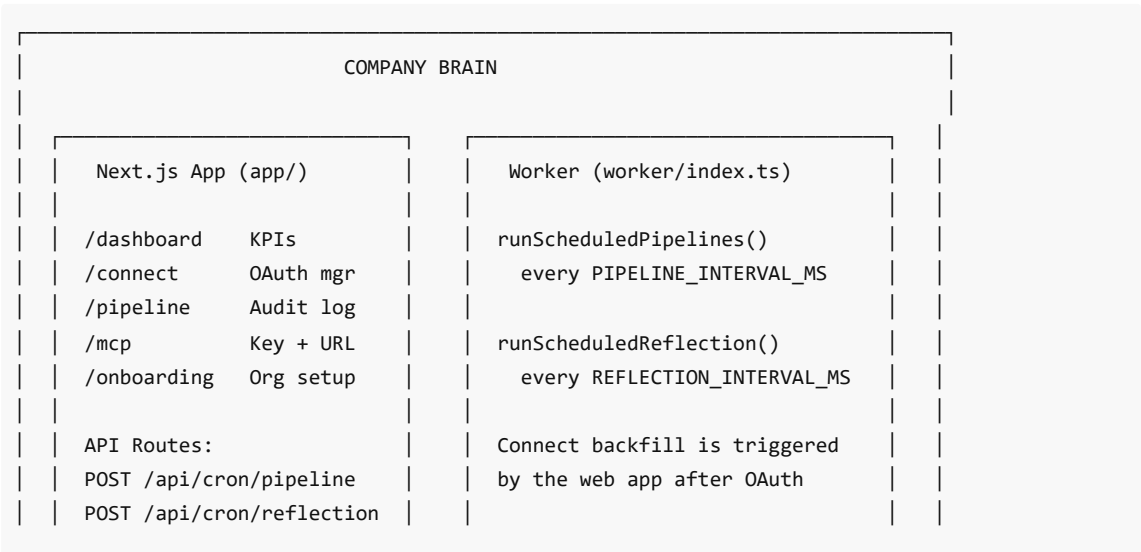
Each pipeline run calls ingestion → extraction in sequence, with up to 40 extraction batches per run:

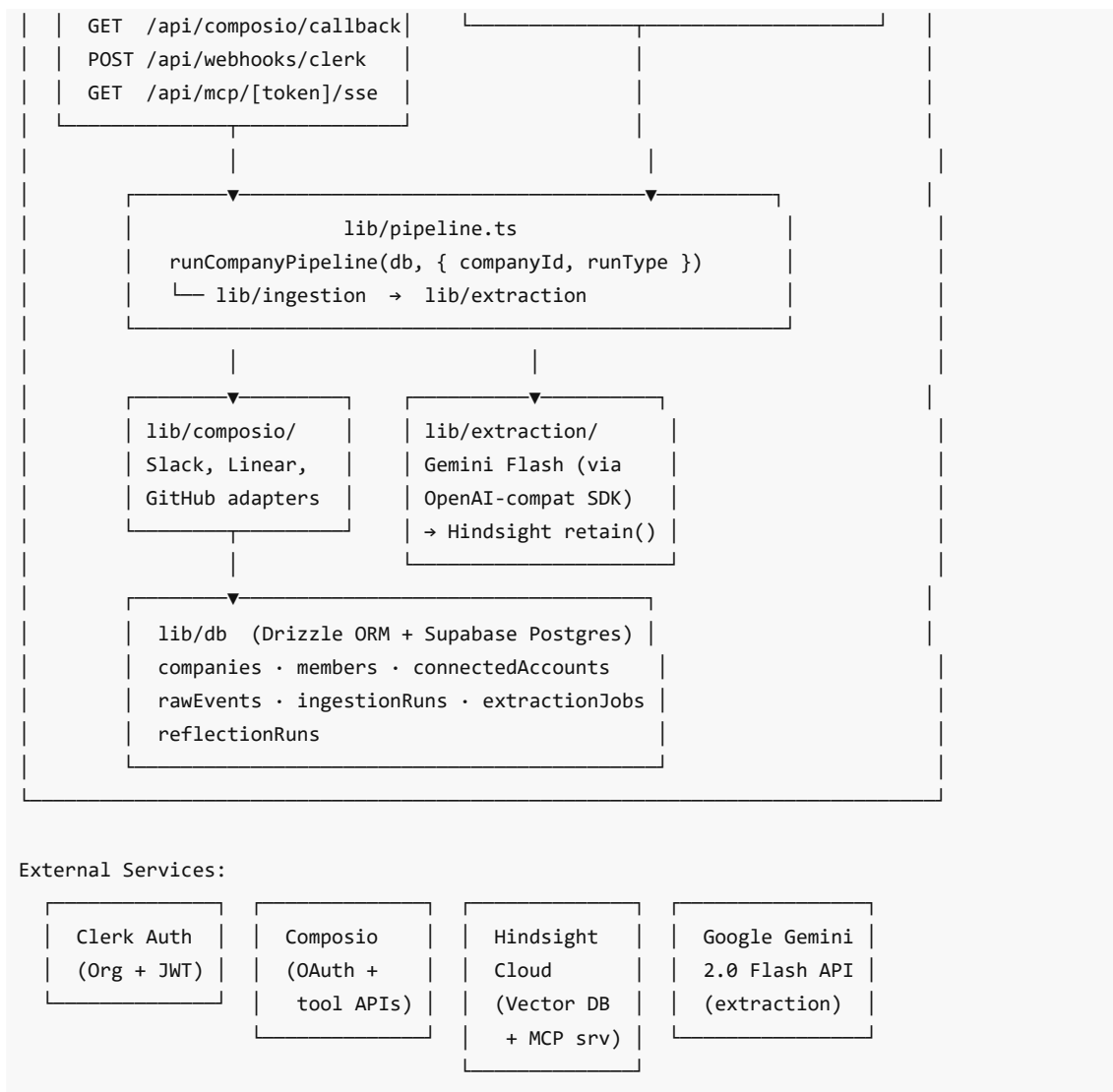
```
// lib/pipeline.ts
const result = await runCompanyPipeline(db, {
  companyId: "your-company-uuid",
  runType: "incremental", // or "backfill" for a 48-hour history pull
});
// result.extraction = { processed, skipped, failed }
```

Gemini classifies each raw event into typed knowledge items:

```
{
  type: "fact" | "decision" | "risk" | "action_item" | "milestone",
  content: "string – standalone, self-contained statement",
  confidence: 0.0-1.0,
  entities: ["ProjectX", "Alice", "auth-service"]
}
```

Architecture





Tech stack

Layer	Technology
Web application	Next.js 16, React 19, Tailwind CSS v4
Pipeline worker	Node.js, tsx
Authentication	Clerk (organization-scoped, webhooks for member sync)
Database	Supabase Postgres, Drizzle ORM, SQL migrations
Tool connectivity	Composio (Slack, GitHub, Linear adapters)
AI extraction	Google Gemini 2.0 Flash via OpenAI-compatible SDK
Vector memory	Hindsight Cloud (retain, reflect, mental models, MCP)

Data flow

A pipeline run begins when the worker (or a manual cron trigger) calls `runCompanyPipeline`. The ingestion stage fetches raw events from every active connected account via Composio, deduplicates them with a SHA-256 key, and writes them to `raw_events`. The extraction stage processes each `pending` event in batches: Gemini classifies each into typed knowledge items, which are immediately retained in the company's isolated Hindsight Memory Bank.

At the reflection interval, five leadership-oriented queries run against the bank; the resulting observations are re-retained as dated documents (`reflection:{key}:{YYYY-MM-DD}`) and logged to `reflection_runs`. The MCP proxy at `/api/mcp/{token}/sse` makes the bank queryable by any MCP-compatible AI client without exposing Hindsight credentials.

Configuration

Copy `.env.example` to `.env` and populate every required variable before starting.

Variable	Required	Default
NEXT_PUBLIC_APP_URL		http://localhost:3000
NODE_ENV		development
CRON_SECRET	 (prod)	—
DEFAULT_COMPANY_ID	—	—
PIPELINE_INTERVAL_MS	—	3600000 (1 h)
REFLECTION_INTERVAL_MS	—	86400000 (24 h)
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY		—

CLERK_SECRET_KEY	✓	—
CLERK_WEBHOOK_SIGNING_SECRET	✓	—
DATABASE_URL	✓	—
DATABASE_DIRECT_URL	—	—
COMPOSIO_API_KEY	✓	—
COMPOSIO_AUTH_CONFIG_*	—	—
HINDSIGHT_API_URL	✓	https://api.hindsight.vectorize.io
HINDSIGHT_API_KEY	✓	—
HINDSIGHT_SCOPED_KEY_EXPIRES_IN_DAYS	—	365

GEMINI_API_KEY	✓	—
GEMINI_MODEL	—	gemini-2.0-flash
GEMINI_API_BASE_URL	—	https://generativelanguage.googleapis.com/v1beta/openai/

API Reference

POST /api/cron/pipeline

Manually trigger an incremental pipeline for a company. Protected by `CRON_SECRET` in production.

Request

```
{ "companyId": "uuid-of-the-target-company" }
```

Response 200

```
{
  "success": true,
  "runId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "extraction": { "processed": 42, "skipped": 8, "failed": 0 }
}
```

Response 401 — Missing or invalid `Authorization: Bearer <CRON_SECRET>` (production only).

Response 400 — `companyId` was not provided in the body, query string, or `DEFAULT_COMPANY_ID`.

POST /api/cron/reflection

Manually trigger the nightly reflection pass for a company. Same auth rules as the pipeline route.

Request

```
{ "companyId": "uuid-of-the-target-company" }
```

Response 200

```
{
  "success": true,
  "runsCount": 5,
  "errors": []
}
```

GET /api/composio/callback

OAuth redirect handler. Composio redirects here after a member authorizes a tool connection. Triggers a connect backfill pipeline for the newly connected toolkit only.

POST /api/webhooks/clerk

Clerk webhook endpoint for organization membership sync. Handles `organizationMembership.created` and `organizationMembership.deleted` events. Requires `CLERK_WEBHOOK_SIGNING_SECRET`.

GET /api/mcp/{token}/sse

MCP SSE proxy to the company's Hindsight memory bank. Authenticated by the per-company `mcpToken` UUID — no separate API key required.

- **GET** — Opens an SSE stream; rewrites Hindsight `endpoint` events to point at `/api/mcp/{token}/message`
- **POST** — Forwards Streamable HTTP JSON-RPC requests to Hindsight
- **OPTIONS** — CORS preflight

Copy the full URL from `/mcp` in the dashboard (e.g. `https://your-app.com/api/mcp/{token}/sse`).

POST /api/mcp/{token}/message

JSON-RPC message endpoint for MCP clients using the SSE transport. Same token authentication as the SSE route.

Contributing

1. Fork the repository and clone your fork
2. Branch off `main` — use `feat/`, `fix/`, or `chore/` prefixes
3. Commit with [Conventional Commits](#)
4. Open a pull request against `main`

See [CONTRIBUTING.md](#) for local setup, project layout, and PR checklist.

Roadmap

- ☒ Multi-tenant Clerk organization authentication
- ☒ Composio OAuth adapters: Slack, GitHub, Linear
- ☒ Gemini-powered signal extraction with Zod validation
- ☒ Hindsight Memory Bank provisioning with mental models
- ☒ Scheduled pipeline worker (incremental + backfill)
- ☒ Nightly reflection with observation re-retention

- ☒ Token-based MCP proxy and scoped API key minting
 - ☒ Pipeline audit dashboard with extraction metrics
 - ☐ Microsoft Teams and Gmail ingestion adapters
 - ☐ Webhook-driven real-time ingestion (vs. polling)
 - ☐ Slack notification integration for reflection summaries
 - ☐ Multi-region deployment support
 - ☐ Self-hosted Hindsight backend option
-

Acknowledgments

- [Hindsight / Vectorize](#) — vector memory bank, mental models, MCP server, and scoped API key infrastructure
 - [Composio](#) — OAuth tool connectivity for Slack, GitHub, and Linear
 - [Google Gemini](#) — `gemini-2.0-flash` powers the organizational signal extraction pipeline
 - [Clerk](#) — organization-scoped authentication and webhook-based provisioning
 - [Supabase](#) — managed Postgres with connection pooling
 - [Drizzle ORM](#) — type-safe schema and relational queries
 - [Next.js](#) — App Router web framework
-

Made with care by the Company Brain team